Practical - 6

Practical Title: Rendering External JSON Data using Fetch API, Search and Filter

Problem Definition: Display student profiles from an external JSON file using the Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

Theory :

1. How is JSON fetched, parsed, and rendered?

The fetch("students.json") method sends a request and returns a promise. The first .then() calls response.json() to parse the JSON text into a JavaScript array. The second .then() stores the array in the students variable and calls displayStudents(), which loops over the records and creates a card (div) for each student using a template literal, then adds it to the page with appendChild().

2. Which array methods are used for search, filter, sort, and pagination?

filter() is used for search (name or email with includes()) and for the course filter. sort() with localeCompare() sorts by name, and sort() with a subtraction compares marks high-to-low or low-to-high. slice(start, end) gives the 4 students of the current page for pagination, forEach() renders the cards, and the spread operator [...students] makes a copy of the data.

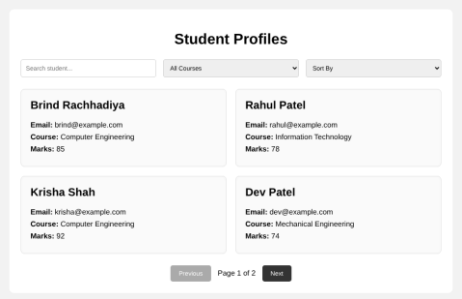
3. How are loading and error states handled?

Errors are handled with .catch(), which shows the message "Unable to load student data." on the page and logs the error in the console. If no student matches the search or filter, the message "No students found." is shown. The Previous and Next buttons are disabled on the first and last page. A "Loading..." message can be shown before the data arrives for a better experience.

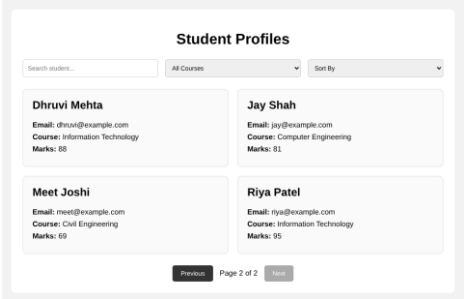
4. How is modularity maintained in JavaScript files?

The project is separated into index.html (structure), style.css (design), script.js (logic), and students.json (data), so data can change without touching the code. Inside script.js, each task has its own function: displayStudents() renders, applyFilters() handles search, filter, and sort, and updatePagination() updates the buttons and page number. Event listeners are kept separate from these functions.

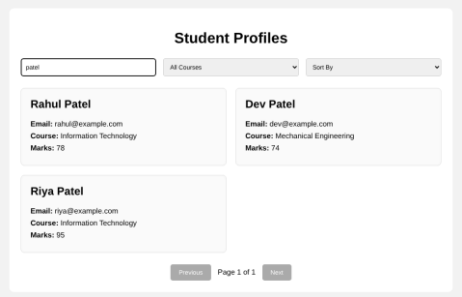
Implementation/ Output/Result



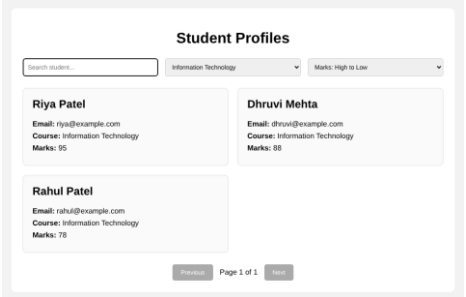
1



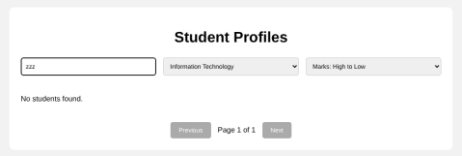
2



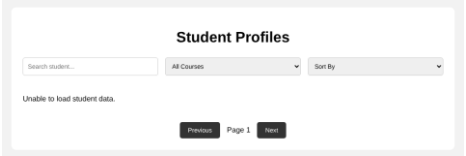
3



4



5



6

Figure: (1) page 1 of students, (2) page 2 using pagination, (3) search for "patel", (4) Information Technology filter with marks high to low, (5) no result found, (6) error message when JSON cannot be loaded.

GitHub Repository Link:

https://github.com/rachhadiyabrind-del/WDF_B/tree/main/Practical%206

Learning Outcomes:

- Fetch external JSON data using the Fetch API
- Parse JSON and render it dynamically on the page
- Implement search and filter using array methods
- Sort data by name and marks
- Build pagination logic for large data lists
- Handle error and empty-result states in the UI

Faculty Remarks: (Keep this blank for faculty)